TP4 : Les DataFrames PySpark — Exercice 1

Cours : Processus décisionnel BigData (EMSI) | **Enseignante :** Pr. BENHADDI Meriem
Objectif : Guide pas à pas des solutions pour l'Exercice 1 (Partie I & Partie II)

PySpark Guide

> 📌 **Contexte et Structure du TP4**
>
> Le sujet de TP4 contient deux exercices distincts numérotés **Exercice 1** :
>
> - **Partie I - Exercice 1 :** `spark.createDataFrame` — Création d'un DataFrame à partir d'une collection Python (5 méthodes).
> - **Partie II - Exercice 1 :** Transformations — Sélection et filtre simples sur une table d'employés.

0. Initialisation de la SparkSession

Point d'entrée obligatoire pour instancier PySpark et créer le contexte de travail :

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TP4_Ex1") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext  # Nécessaire pour la méthode RDD
```

I. Partie 1 — Exercice 1 : `spark.createDataFrame` (5 méthodes)

a) Depuis une liste de tuples

Les données sont passées sous forme de tuples bruts avec la liste explicite des noms de colonnes :

```
donnees = [(1, "Alice", 30), (2, "Bob", 25), (3, "Charlie", 35)]
df = spark.createDataFrame(donnees, ["id", "nom", "age"])
df.show()
```

```
+---+-------+---+
| id|    nom|age|
+---+-------+---+
|  1|  Alice| 30|
|  2|    Bob| 25|
|  3|Charlie| 35|
+---+-------+---+
```

b) Depuis une liste de dictionnaires (schéma auto-déduit)

Spark inspecte les clés des dictionnaires pour nommer automatiquement les colonnes :

```
donnees = [{"id": 1, "nom": "Alice"}, {"id": 2, "nom": "Bob"}]
df = spark.createDataFrame(donnees)
df.show()
```

c) Depuis une liste d'objets `Row`

Row représente une ligne structurée type "tuple nommé" avec des champs nommés :

```
from pyspark.sql import Row

rows = [Row(id=1, nom="Alice"), Row(id=2, nom="Bob")]
df = spark.createDataFrame(rows)
df.show()
```

d) Depuis un RDD existant

Conversion d'un RDD Spark (créé avec `parallelize`) vers un DataFrame :

```
rdd = sc.parallelize([(1, "Alice"), (2, "Bob")])
df = spark.createDataFrame(rdd, ["id", "nom"])
df.show()
```

e) Avec un schéma explicite (Types et Nullabilité contrôlés)

Définir le schéma évite l'inférence automatique lente et garantit l'intégrité des données :

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType

schema = StructType([
    StructField("id", IntegerType(), False),  # False: NON NULL
    StructField("nom", StringType(), True)    # True: Valeur NULL acceptée
])

df = spark.createDataFrame([(1, "Alice"), (2, "Bob")], schema=schema)
df.printSchema()
```

```
root
 |-- id: integer (nullable = false)
 |-- nom: string (nullable = true)
```

II. Partie 2 — Exercice 1 : Sélection et filtre simples

> **Énoncé de l'exercice :** Affiche le nom et le salaire des employés qui gagnent **plus de 40 000**, triés par salaire **décroissant**.

1. Jeu de données initial

```
from pyspark.sql import functions as F
from pyspark.sql.types import StructType, StructField, IntegerType, StringType

data = [
    (1, "Alice", "Paris", "Informatique", 45000, "2020-01-15"),
    (2, "Bob", "Lyon", "Ventes", 38000, "2019-03-22"),
    (3, "Charlie", "Paris", "Informatique", 52000, "2018-07-10"),
    (4, "Diana", "Marseille", "Ventes", 41000, "2021-11-05"),
    (5, "Eve", "Lyon", "RH", 36000, "2020-05-30"),
    (6, "Frank", "Paris", "Informatique", 60000, "2017-02-14"),
    (7, "Grace", "Marseille", "RH", 39000, "2022-09-01"),
    (8, "Heidi", "Lyon", "Ventes", 42000, None),
]

schema = StructType([
    StructField("id", IntegerType(), False),
    StructField("nom", StringType(), True),
    StructField("ville", StringType(), True),
    StructField("departement", StringType(), True),
    StructField("salaire", IntegerType(), True),
    StructField("date_embauche", StringType(), True),
])

df = spark.createDataFrame(data, schema)
```

2. Requête PySpark

```
resultat = df.filter(F.col("salaire") > 40000) \
             .select("nom", "salaire") \
             .orderBy(F.desc("salaire"))

resultat.show()
```

3. Décomposition des étapes

Méthode PySpark	Équivalent SQL	Explication
`df.filter(F.col("salaire") > 40000)`	`WHERE salaire > 40000`	Filtre les lignes pour ne garder que celles où le salaire dépasse 40 000.
`.select("nom", "salaire")`	`SELECT nom, salaire`	Projection : conserve uniquement les colonnes demandées.
`.orderBy(F.desc("salaire"))`	`ORDER BY salaire DESC`	Tri décroissant des résultats selon le montant du salaire.
`.show()`	-	Action déclenchant l'évaluation et affichant le résultat en console.

4. Résultat en sortie

```
+-------+-------+
|    nom|salaire|
+-------+-------+
|  Frank|  60000|
|Charlie|  52000|
|  Alice|  45000|
|  Heidi|  42000|
|  Diana|  41000|
+-------+-------+
```